```python
'''
main.py (Python)
The Accessibility Assistant software allows blind people to give our AI assistant a prompt and have the AI act directly on the user's computer. The person who is blind could type onto the auto-focused textbox using touch typing, their device's built-in dictation (speech-to-text), or a braille keyboard, among other solutions. The software can use TTS to communicate directly with the user, so it is recommended, but not required, to raise your volume. Closing the application will halt AI action.
Notes:
Please run 'pip install customtkinter pyautogui keyboard pyttsx3 mss Pillow anthropic SpeechRecognition pyaudio' in your terminal before using the software to ensure function.
If needed, please give the program any requested permissions (including screen recording, accessibility, etc.). It is necessary for software function.
'''

import customtkinter as ctk
import pyautogui # Mouse control library
import keyboard # Keyboard control library
import mss # Screenshot library
import mss.tools
from PIL import Image, ImageDraw, ImageFont # Annotates captured screenshot for LLM
import time
import base64 # For inputting image into LLM
from io import BytesIO # For inputting image into LLM
import anthropic # For querying LLM
import tkinter as tk # For GUI
import platform # For detecting OS
import random
import pyttsx3
import speech_recognition as sr
import threading

client = anthropic.Anthropic(
    api_key="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" # Personal API key may be used for testing purposes
)

recognizer = sr.Recognizer()

screen_width, screen_height = pyautogui.size()

# Initialize GUI
root = ctk.CTk()
root.title("")
root.attributes('-alpha', 0.9)

window_corner_radius = 0
if platform.system() == "Windows":
    root.overrideredirect(True)
    root.wm_attributes('-transparentcolor', 'black')
    window_corner_radius = 30
elif platform.system() == "Darwin":
```

```python
    root.resizable(False, False)

# Content frame
content_frame = ctk.CTkFrame(root, corner_radius=window_corner_radius)
content_frame.pack(fill='both', expand=True, padx=2, pady=2)

# Upper frame
upper_frame = ctk.CTkFrame(content_frame, fg_color="transparent")
upper_frame.pack(side='top', fill='x', padx=2, pady=2)

# Heading frame
heading_frame = ctk.CTkFrame(upper_frame, fg_color="transparent")
heading_frame.place(relx=0.5, rely=0.5, anchor='center')

# Heading label
heading_label = ctk.CTkLabel(heading_frame, text="Blind Trust", font=ctk.CTkFont(weight="bold"))
heading_label.pack(side='left', pady=5)

# Button frame
button_frame = ctk.CTkFrame(upper_frame, fg_color="transparent")
button_frame.pack(side='right', padx=2)

# Close button
if platform.system() == "Windows":
    close_button = ctk.CTkButton(button_frame, text="×", command=root.destroy, fg_color="transparent", text_color="black", width=20)
    close_button.pack(side='left', padx=5, pady=2)

# Prompt textbox
prompt_entry = ctk.CTkEntry(content_frame, placeholder_text="Type a prompt or press space to talk...", width=40*8)
prompt_entry.pack(side='left', padx=5, pady=2)

def query_agent(event=None):
    # Queries LLM agent to complete user request
    user_prompt = prompt_entry.get()

    # Focuses away from GUI
    if platform.system() == "Darwin":
        keyboard.press_and_release('cmd+tab')
    else:
        keyboard.press_and_release('alt+tab')

    # GUI disappears
    root.withdraw()

    time.sleep(0.25)

    def capture_screen(monitor_number=1, grid_spacing=50):
        # Captures user's screen and overlays grid for LLM reference
        with mss.mss() as sct: # SCT = Screen capture tool
            monitor = sct.monitors[monitor_number]
            sct_img = sct.grab(monitor)

            # Set up grid overlay
```

```
            img = Image.frombytes("RGB", sct_img.size, sct_img.rgb)
            draw = ImageDraw.Draw(img)
            font = ImageFont.load_default()

            width, height = img.size

            # Create grid overlay
            for x in range(0, width, grid_spacing):
                draw.line([(x, 0), (x, height)], fill="gray", width=1)
                draw.text((x + 2, 5), str(x), fill="gray", font=font)

            for y in range(0, height, grid_spacing):
                draw.line([(0, y), (width, y)], fill="gray", width=1)
                draw.text((5, y + 2), str(y), fill="gray", font=font)

            # img.save("screenshot_with_grid.png") # For testing
            # img.show() # For testing
            return img

    def img_to_base64(img, format="PNG"):
        # Reformats image for LLM
        buffered = BytesIO()
        img.save(buffered, format=format)
        img_bytes = buffered.getvalue()
        encoded_bytes = base64.b64encode(img_bytes)
        encoded_string = encoded_bytes.decode('utf-8')
        return encoded_string

    command_cycles = 0
    past_messages = []

    while command_cycles < 20:
        # Accomplishes tasks as given by LLM commands with less than 20 cycles of LLM IO
        print("Starting command cycle", command_cycles+1)
        screen_image_data = img_to_base64(capture_screen(monitor_number=1, grid_spacing=25))

        # Requests commands from LLM
        llm_message = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=1024,
            system="You are an AI agent for a blind user that can take action on the user's screen. YOU MUST REPLY ONLY WITH A SERIES OF COMMANDS, all in one line and each started with TWO exclamation points (!!) and terminated with TWO semicolons each (;;), to take action. You are provided an image of the user's screen with a precise grid with 25px subdivisions. Screen dimensions are " + str(screen_width) + "x" + str(screen_height) + ". Commands, in format !!COMMAND(PARAM1, PARAM2);;, include !!CLICK(COORDX, COORDY);;, !!WRITE(`STRING`);;, !!PRESS(`KEY`);;, !!SCROLLUP();;, !!SCROLLDOWN();;, !!PAUSE(SECONDS);;, !!MESSAGE(`TEXT`);;, !!REVIEW();;, !!END();;. CLICK and WRITE allow you to control the user's screen. PRESS lets you press and release a key or shortcut, with keys separated with a + (use shortcuts over clicks when possible). PAUSE lets you delay actions (short delays between actions recommended). MESSAGE lets you tell the user something or give them feedback during your process. REVIEW is to be used for complex tasks where changes on the screen as expected, as it reprompts you with the user's updated screen after prior actions before terminating the
```

```
current string of commands. END marks the end of your task and terminates the string of
commands, and is recommended for one-step tasks. Anything you send outside of commands
will NOT be read to the user. Make sure to have any text you would like the user to see
in the MESSAGE command. ONCE AGAIN, text outside of commands will result in the your
FAILURE as an agent. Notes: ALWAYS !!PRESS(`win+s`);; or !!PRESS(`cmd+space`);; to
search for and open applications on different OSs, instead than clicking on tiny icons
(which you WILL miss). If navigating to a website or new page, be sure to REVIEW and
PAUSE signfiicantly before continuing to allow for loading times. And, if searching for
something, use browser and built-in shortcuts rather than clicking on any searchbar. If
you cannot see your target clearly or it is near the very top/bottom of the screen,
SCROLLUP AND SCROLL DOWN are small bursts of scrolling, equivalent to `page up` and
`page down`. If you are trying to click a button and it didn't work the first time, use
!!PRESS(`tab`);; to cycle through elements on screen and press enter. Even if the user
has not reprompted you, and the last message was from the assistant, keep providing
commands. You will not be prompted if there was nothing to do, and END terminates your
cycles. Make sure to click the middle of buttons, not the top left corner. If the
screen isn't changing when you click a button, try offsetting your click by a good bit
(often to the down and right), as you may be missing slightly.",
            messages= past_messages + [
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "image",
                            "source": {
                                "type": "base64",
                                "media_type": "image/png",
                                "data": screen_image_data,
                            },
                        },
                        {
                            "type": "text",
                            "text": user_prompt
                        }
                    ],
                }
            ],
        )

        # print(llm_message.content[0].text) # For testing/debugging

        # Saves actions to give LLM memory between command cycles
        past_messages.append({"role": "assistant", "content": [{"type": "text", "text":
llm_message.content[0].text}]})
        past_messages.append({"role": "user", "content": [{"type": "text", "text":
"Continue the task provided given your previous progress."}]})

        # Parses commands
        commands = llm_message.content[0].text.split(";;")[:-1]
        commands = [command.strip() for command in commands]
        commands[0] = commands[0][commands[0].index("!!"):]

        # Executes all commands
        review = False
        for command in commands:
            time.sleep(0.1)
```

```
            if command.startswith("!!CLICK"):
                coords = command[(command.index('(') + 1):command.index(')')].split(',')
                coords = [int(coord) for coord in coords]
                pyautogui.click(coords[0]+random.randint(-20, 50), coords[1]+random.randint(-20, 50))
            elif command.startswith("!!WRITE"):
                text = command[(command.index('(`') + 2):command.index('`)')]
                keyboard.write(text)
            elif command.startswith("!!PRESS"):
                keys = command[(command.index('(`') + 2):command.index('`)')]
                keyboard.press_and_release(keys)
            elif command.startswith("!!SCROLLUP"):
                keyboard.press_and_release('page up')
            elif command.startswith("!!SCROLLDOWN"):
                keyboard.press_and_release('page down')
            elif command.startswith("!!PAUSE"):
                delay = float(command[(command.index('(') + 1):command.index(')')])
                time.sleep(delay)
            elif command.startswith("!!MESSAGE"):
                text = command[(command.index('(`') + 2):command.index('`)')]
                print("Message received:", text)
                pyttsx3.speak(text)
            elif command.startswith("!!REVIEW"):
                review = True
                time.sleep(3)
                break
            elif command.startswith("!!END"):
                break

        command_cycles += 1
        if not review:
            break

    # GUI reappears
    root.deiconify()

continue_speech = False
def speech_to_text(event=None):
    prompt_entry.configure(placeholder_text="Listening...", state='disabled')
    root.update()

    def listen():
        voice_button_color = voice_button.cget('fg_color')
        voice_button_hover_color = voice_button.cget('hover_color')
        try:
            with sr.Microphone() as source:
                recognizer.adjust_for_ambient_noise(source, duration=0.3) # Calibrate to background noise
                audio = recognizer.listen(source, timeout=5, phrase_time_limit=30) # Stops after 5 second pause or 15 seconds of speech

            text = recognizer.recognize_google(audio)
            global continue_speech
            continue_speech = True

```

```python
            # Update UI on main thread
            root.after(0, lambda: [
                prompt_entry.configure(state='normal', placeholder_text="Type a prompt or press space to talk ... "),
                prompt_entry.insert('end', text + ' '),
                prompt_entry.focus_set(),
                voice_button.configure(fg_color=voice_button_color, hover_color=voice_button_hover_color)
            ])

        except sr.WaitTimeoutError:
            root.after(0, lambda: prompt_entry.configure(state='normal', placeholder_text="Type a prompt or press space to talk ... "), voice_button.configure(fg_color=voice_button_color, hover_color=voice_button_hover_color))
        except sr.UnknownValueError:
            root.after(0, lambda: prompt_entry.configure(state='normal', placeholder_text="Couldn't hear anything, try again ... "), voice_button.configure(fg_color=voice_button_color, hover_color=voice_button_hover_color))
        except sr.RequestError:
            root.after(0, lambda: prompt_entry.configure(state='normal', placeholder_text="Speech service unavailable ... "), voice_button.configure(fg_color=voice_button_color, hover_color=voice_button_hover_color))

    threading.Thread(target=listen, daemon=True).start()
    voice_button.configure(fg_color=["#F13636", "#8E1C1C"], hover_color="#8E1C1C")


# Enter button
enter_button = ctk.CTkButton(content_frame, text="➢", width=20, command=query_agent)
enter_button.pack(side='right', padx=5, pady=3)

# STT button
voice_button = ctk.CTkButton(content_frame, text="🎙", width=20, command=speech_to_text)
voice_button.pack(side='right', pady=3)

# Makes window draggable
root._drag_x = None
root._drag_y = None

def initialize_drag(event):
    if isinstance(event.widget, tk.Entry):
        root._drag_x = None
        root._drag_y = None
        return "break"
    root._drag_x = event.x
    root._drag_y = event.y

def execute_drag(event):
    if root._drag_x is None or root._drag_y is None:
        return
    x = root.winfo_x() + event.x - root._drag_x
    y = root.winfo_y() + event.y - root._drag_y
```

```
    root.geometry(f"+{x}+{y}")

root.bind("<ButtonPress-1>", initialize_drag)
root.bind("<B1-Motion>", execute_drag)
def on_key_press(event):
    if event.char:
        global continue_speech
        if event.char ≠ ' ': continue_speech = False
        if str(root.focus_get()) ≠ ".!ctkframe.!ctkentry.!entry":
            if event.char == ' ':
                speech_to_text()
            else:
                prompt_entry.insert('end', event.char)
                prompt_entry.focus_set()
    elif event.keysym == "Enter" or event.keysym == "Return":
        query_agent()
root.bind("<Key>", on_key_press)
root.bind("<Alt-m>", speech_to_text)
root.bind("<Control-m>", speech_to_text)

root.mainloop()
```